实验 10 基于原始套接字实现 ping 命令

实验目的:

1.掌握原始套接字编程技术。

2.了解 ICMP 协议，掌握利用 ICMP 协议测试网络连通性的工作原理。

实验环境:

1.联网并运行 Linux 操作系统的 PC 机

实验内容和要求:

基于原始套按字技术，实现满足如下要求的网络应用程序：

1.使用 ICMP 协议中的 Echo Request/Echo Reply 报文对，测试到目的主机的网络连通性。要求显示的内容有：目的主机的 IP 地址、接收到 ICMP Echo Reply 报文的长度、从源主机到目的主机的往返时间（以 ms 为单位）、接收到报文的 TTL 值。

2.目的主机地址既可以使用点分十进制形式 IP 地址给出，也可以使用域名形式给出。

实验原理

1、关于 ICMP 协议

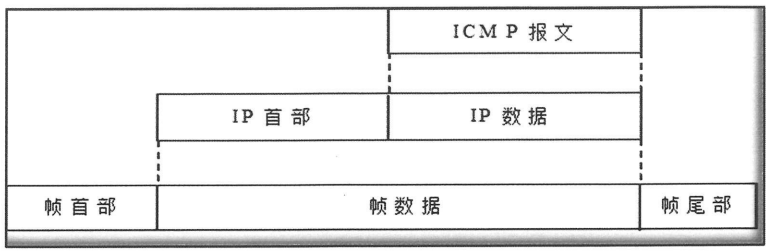
（1）ICMP 简介

因特网控制报文协议(Internet Control Message Protocol，ICMP)是 IP 协议的补充，它的主要功能是传递控制信息及提供差错报告。用于 IP 层的差错报告、控制以及路由器或主机信息的获取。

ICMP 协议设计的最初目的主要是用于 IP 层的差错报告，由路由器或信宿以一对一的模式向信源报告传输错误的原因。随着网络的发展，检测和控制功能逐渐被引入到 ICMP 协议中，使得 ICMP 协议不仅用于传输差错报告，而且大量用于传输控制报文。

ICMP 协议运行于 IP 协议之上，其报文封装在 IP 数据报的数据部分进行传输，如下图所示。封装了 ICMP 报文的 IP 首部中的协议号为 1。

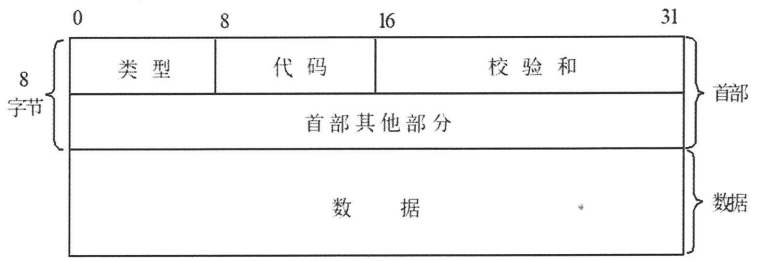
虽然 ICMP 报文封装在 IP 数据报中传输，但是，这并不意味着 ICMP 是比 IP 更高的层次。因为，ICMP 软件只是 IP 软件的一个组成部分。它在概念上并不构成一个独立的层次，只是解决了互联网中的一类特殊问题，为 IP 协议提供服务。所以，通常被认为 ICMP 是 IP 协议的一部分。



ICMP 报文由首部和数据段组成。首部为定长的 8 个字节，前 4 个字节是通用部分，后 4 个字节随报文类型的不同有所差异。数据段的长度可变。如下图所示。

其中：

- 类型(Type)字段长度为一个字节，指出 ICMP 报文的类型。
- 代码(Code)字段长度为一个字节，提供关于报文类型的进一步信息。
- 校验和字段(Checksum)长度为两个字节，对整个 ICMP 报文进行校验，校验和的计算方法与 IP 数据报的相同。
- 首部其他部分为 4 个字节，大部分差错报告报文未用到这一部分。参数错报告报文用到其中的一个字节作指针；请求应答报文对利用这 4 个字节匹配请求与应答报文。不使用时将不使用部分填“0”。
- 数据段部分：在报告差错时，携带原始出错数据报的首部和数据的前 8 个字节，通常这些信息包括了该数据报的关键信息（前 8 个字节一般为上层协议的首部信息）。在请求/应答报文中，携带相关的额外信息。



ICMP 报文按照其作用分为三类：差错报告报文、控制报文和请求应答报文。

（2）ICMP 请求与应答报文对

ICMP 请求与应答报文对的出现使得因特网上的任何主机或路由器可以向其他主机或路由器发送请求并获得应答。通过 ICMP 请求与应答报文对，网络管理人员、用户或应用程序可以对网络进行检测，目的是利用这些有用的信息，对网络进行故障诊断和控制。

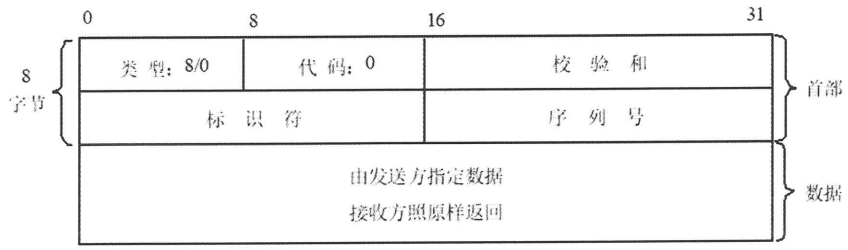
ICMP 请求与应答报文对如下表所示。其中的信息请求与应答报文已经不再使用。

类型	作用	代码	报文
8	回应请求与应答	0	回应请求
0		0	回应应答
10	路由器请求与通告	0	路由器请求
9		0	路由器通告

13	时间戳请求与应答	0	时间戳请求
14		0	时间戳应答
15	信息请求与应答（已不用）	0	信息请求
16		0	信息应答
17	地址掩码请求与应答	0	地址掩码请求
18		0	地址掩码应答

回应请求与应答报文的目的是对网络进行诊断和测试。它不仅可以被用来测试主机或路由器的可达性，还可以测试 IP 协议的工作情况。TCP/IP 网络系统所提供的 ping 命令大多是利用 ICMP 回应请求与应答报文来实现的，该命令通常用于测试信宿的可到达性。

ICMP 回应请求与应答报文的格式如图所示：



其中：类型“8”表明是回应请求报文，类型“0”表明是回应应答报文。标识符和序列号通常用于匹配请求与应答。标识符一般为发起请求进程的进程 ID，或者为标识会话的端口号。序列号在多个回应请求中会自动累加。回应请求与应答报文的标识符和序列号一致。

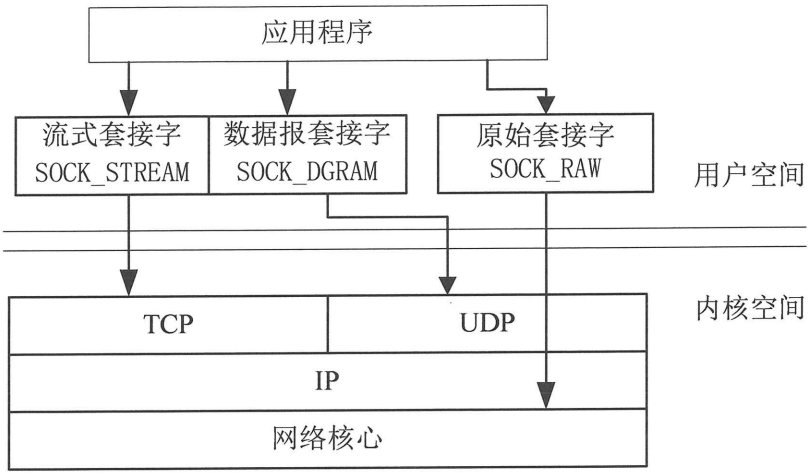
2、原始套接字编程

（1）原始套接字编程接口简介

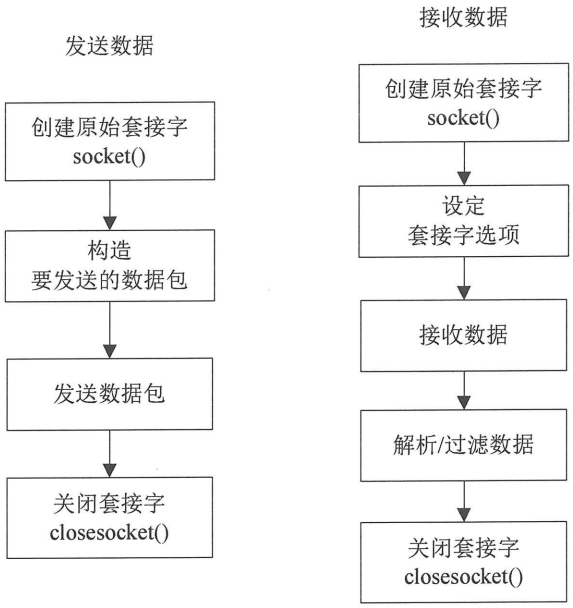
原始套接字是在传输层之下使用的套接字编程接口，该编程接口被广泛应用于高级网络编程，是黑客常用的一种编程手段，像是著名的网络抓包工具 sniffer、拒绝服务攻击（DOS）、IP 欺骗等都可以使用原始套接字编程实现。原始套接所提供的特殊功能主要包括：

- 可对 ICMP、IGMP 等较低层次的协议直接访问，直接发送或接收 ICMP、IGMP 等协议的报文。
- 能够接收一些 TCP/IP 栈不能处理的特殊的 IP 分组，通常是网络核心不能识别 IP 分组的协议字段。
- 通过设置原始套接字的 IP_HDRINCL 选项，可以发送用户自定义分组首部的 IP 分组，因此可编写测试基于 IP 的高层网络协议。
- 可以通过原始套接字将网卡设置为混杂模式，从而能够接收所有流经网卡接口的 IP 分组，达到进行网络监听的目的。

原始套接字与流式套接字和数据报套接字编程接口的对比如下图所示。由图中可见:流式套接字和数据报套接字分别“悬浮”于 TCP 协议和 UDP 协议的外围，只能通过这两个传输层协议完成数据通信功能。而原始套接字直接置“根”于操作系统内核的网络核心（Network Core），因而可对 IP 分组直接操控。



原始套接字的通信是基于 IP 分组的，因而是无连接、不可靠的，其编程流程与数据报套接字类似。其编程框架如下图所示。



使用原始套接字编程通常较为复杂，这是因为，发送数据时要进行复杂的数据包首部字段的构造与填写，接收数据时又涉及数据包首部中各字段的分析解释。需要编程者对 TCP/IP 协议的原理有较为深入的理解。

（2）原始套接字编程接口函数的使用

创建原始套接字所使用的函数仍然是：socket()，但用于指定套接字类型的第二个参数应使用 SOCK_RAW。比如：

int sockRAW=socket(AF_INET, SOCK_RAW, protocol);

其中第三个参数 protocol 用来指明创建的原始套接字所能接收或发送的 IP 分组的协议字段的协议类型，可取如下值：

- IPPROTO_ICMP：表示 ICMP 协议。

- IPPROTO_IGMP：表示 IGMP 协议。
- IPPROTO_TCP：表示 TCP 协议。
- IPPROTO_UDP：表示 UDP 协议。
- IPPROTO_IP：原始 IP 协议，该类原始套接字可用于接收任何的 IP 数据包。
- IPPROTO_RAW：原始 IP 协议，该类 socket 只能用来发送 IP 包，而不能接收任何的数据。发送的数据需要自己填充 IP 包头，并且自己计算校验和。

创建一个原始套接字时，如果将参数 protocol 指定为 IPPROTO_RAW，则所创建的原始套接字只能用来发送 IP 包，不能接收任何数据，而且要发送的数据需要自己填充 IP 首部。如果将参数 protocol 指定为 IPPROTO_IP，则所创建的原始套接字可用于接收任何 IP 分组，但 IP 分组的校验和验证和协议分析等需要由程序自己完成。

如果参数 protocol 指定为 IPPROTO_ICMP 或 IPPROTO_UDP 等这种即不是 IPPROTO_IP（值为 0）也不是 IPPROTO_RAW（值为 255）的值：**发送数据**时，系统将会按照该参数指定的协议类型自动构造 IP 分组首部，而不用自己填充。**接收数据**时，系统只会将首部协议字段值和该参数值相同的 IP 分组交给该原始套接字。因此，一般来说，要想接收或发送哪个协议的数据包，就应该在创建套接字时将参数 protocol 指定为哪个协议。

例如，使用如下代码创建的原始套接字 sockRaw 只能发送或接收 ICMP 数据包。

sockRaw=socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);

与数据报套接字相似，使用原始套接字发送或接收数据一般要分别使用 sendto()函数和 recvfrom()函数。但如果事先调用 connect()函数绑定了通信对方的 IP 地址，也可以使用 send()函数和 recv()函数来收发数据。

发送数据时，如果没有设置原始套接字的 IP_HDRINCL 选项，并且创建套接字时指定的协议类型不是 IPPROTO_RAW，系统将根据创建套接字时 protocol 参数指定的协议类型来自动构造 IP 分组的首部，程序可填写的部分只是 IP 分组的数据部分。如果已经设置了原始套接字的 IP_HDRINCL 选项，则程序需要自己构造 IP 分组的首部，程序需要填写的内容为包括 IP 首部在内的整个 IP 分组。

在使用原始套接字接收数据时，无论是否设置 IP_HDRINCL 选项，原始套接字收到的数据都是包括 IP 首部在内的完整的 IP 分组，并且首部所有数值字段均为网络字节顺序。

原始套接字可以发送任何载有任何数据的 IP 分组，但是，原始套接字不能接收任何运送 TCP 或 UDP 数据的 IP 分组，这是由于网络核心有专门的 TCP 协议和 UDP 协议模块处理，网络核心不会将收到的运送 TCP 或 UDP 数据的 IP 分组传递给任何原始套接字。如果进程希望读取运送 TCP 或 UDP 数据的 IP 分组，则需要在链路层读取；还有一种方法就是将网卡设置为混杂模式。

原始套接字接收能够接收运送 ICMP 和 IGMP 报文的 IP 分组和系统不能识别其协议字段的 IP 分组。

- 系统在处理完收到的 ICMP 应答请求、ICMP 时间戳请求、ICMP 子网掩码请求等之后，将这些包含 ICMP 包的 IP 分组传递给匹配的原始套接字。
- 对 IGMP 数据包，系统在完成处理后也会将 IP 分组传送给匹配的原始套接字。

- 对于一个收到的不能识别协议字段值的 IP 分组，系统将在检验完 IP 首部中的 IP 版本、头部校验和、头部长度、目的 IP 地址等字段后，也会将该 IP 分组传递给相应的原始套接字。

对收到的每一个能够传递给原始套接字的 IP 分组，系统核心在完成必要的处理之后，将会对所有进程的原始套接字进行检查，每一个匹配的套接字都会收到该 IP 分组的一个副本。系统核心选择匹配套接字的规则如下：

1）创建原始套接字时为原始套接字指定的协议类型必须与 IP 分组的协议字段值匹配，如果原始套接字的协议类型值为 0，则与所有协议类型的 IP 分组相匹配。

2）如果原始套接字已通过调用 connect()函数绑定了通信对端的 IP 地址，则该原始套接字只能接收源地址为该绑定地址的 IP 分组。

3）如果原始套接字已通过调用 bind()函数绑定了本地 IP 地址，则该原始套接字只能接收目的地址为该绑定地址的 IP 分组。

（3）收发 ICMP 数据包

使用原始套接字收发 ICMP 数据包的编程步骤与数据报套接字的编程步骤大体相同，基本流程如下。

1）socket 初始化

2）创建原始套接字，指定协议类型为 IPPROTO_ICMP；

3）指定目的地址（发送）或绑定本地地址（接收）；

4）填充 ICMP 首部（发送）；

5）发送或接收数据；

6）过滤并解析数据包

7）关闭套接字

解析数据包时要根据接收到的 IP 数据报首部长度来计算 ICMP 报文在 IP 数据报中的起始地址，把该地址赋值给类型为 ICMP 报文首部的指针，然后再利用该指针对 ICMP 报文进行解析。

3、ping 程序的实现

Ping 程序主要用来测试两个主机之间的连通性，是一个经典的网络工具程序。该程序使用了 ICMP 的回送请求和应答报文。其基本工作原理如下：

- 源主机向目的主机发送一回送请求报文（类型 8）；
- 如果目的主机收到该报文，目的主机中的 ICMP 模块把运送该报文的 IP 首部中的目的 IP 地址与源 IP 地址交换，并将 ICMP 报文的类型由 8 改为回送回答类型 0，重新计算校验和后再发回源主机；
- 若源主机收到了该回送的报文，则说明了目的主机可达，目的主机与源主机之间的路由器工作正常，源主机和目的主机的 IP、ICMP 软件运行正常。

（1）构造 ICMP 报文

为方便对 ICMP 报文进行处理，可以根据 ICMP 协议的语法定义一个用于封装生成 ICMP 报文的数据结构。可以参考如下代码：

```
#define ICMP_ECHOREPLY 0            /*Echo 应答*/
#define ICMP_ECHO    8              /*Echo 请求*/
/* ICMP消息头部*/
struct icmphdr {
   u8    type;
   u8    code;
   u16   checksum;
   union {
         struct {
               u16   id;
               u16   sequence;
         } echo;
         u32   gateway;
         struct {
               u16   unused;
               u16   mtu;
         } frag;                     /*pmtu发现*/
   }un;
   /*ICMP数据占位符*/
   u8    data[0];
#define icmp_id     un.echo.id
#define icmp_seq    un.echo.sequence
};
#define ICMP_HSIZE    sizeof(struct icmphdr)
```

构造 ICMP 包就是按照 ICMP 包的格式要求填写数据包的各个字段的值。Ping 程序使用的是 ICMP 回送请求和应答报文，这两种报文的格式完全相同。

除了具备所有 ICMP 包都有的类型(type：8bits)、代码(code：8bits)、校验和(checksum：16bits)这三个字段外，回送请求和应答报文首部还有标识符（id：16bits）和序列号（sequence ：16bits）两个字段。ICMP 规定，回送应答报文的标识符、序列号和附加数据必须与回送请求报文相同，由此可以看出，标识符和序列号主要用于区分不同的回送请求报文所对应的回送应答报文。

Ping 程序除了检查网络的连通性外，还可以测量两台主机间的延迟。为了实现这一功能：a)应在 ICMP 回送请求报文的附加数据中保存发送该请求报文的时间戳；b)在应答报文中该时间戳将会被原样返回；c)用接收到回送应答报文时的时间戳减去应答报文中的时间戳所得的值就可以估计两台主机间的延迟。

ICMP 报文中的标识符一般使用进程 ID，通过接收到的 ICMP 报文中的标识符即可判断该报文是不是对本进程发送的 Echo Request 报文的响应。进程 ID 可用如下函数获得：

pid_t getpid(void);

序列号根据实际需要确定的，在 Ping 程序中，由于要连续发送多个 ICMP 回送请求报文，为了表明这多个不同报文的发送顺序，第一个回送请求报文的序列号可设为 0，当然也可以设为 1，随后每发送一个报文，序列号都应加 1。通过序列号可以判断该 Echo Reply 报文是对本进程发送的哪一个请求报文的响应。

时间戳并不是 ICMP 回送请求报文必须的部分，是我们为了计算从发送请求报文开始到收到应答报文所经历的时间而在 ICMP 包的附加数据部分自己添加的。时间戳可用以下函数获得：

gettimeofday(struct timeval *time, NULL);;

ICMP 包的内容除 ICMP 包头外还应包括一部分附加数据，用以调整 ICMP 包的长度，其大小可根据需要而定，其内容根据具体需要来确定。

（2）计算校验和

校验和是在所有 ICMP 包的其他数据填充完后计算填充的。在 TCP/IP 协议族中，计算 IP 分组首部校验和、UDP 和 TCP 数据包的校验和，以及 ICMP 数据包的校验和等，所采用的算法是相同的，不同的只是计算的数据覆盖的范围不同。

使用“反码算术运算”计算校验和的算法如下：

a) 把校验和字段置 0

b) 若待计算校验和的数据长度为奇数个字节，则在数据末尾补一个字节的 0

c) 把数据按 16 位进行累加，最高位产生的进位循环加到和的最低位

d) 把累加和按位取反，即可得到所需的校验和，再把该校验和赋值给校验和字段

“反码校验和算法”有如下特点：

a) 交换性和结合性。

假设用 A、B、C、D、…、Y、Z 代表一系列的 8 位组；使用[A，B]代表：A*256+B。那么，16 位校验和的计算可以表示如下：

[A,B]+[C,D]+…+[Y,Z]

Or:[A,B]+[C,D]+…+[Z,0]

可以下面的形式进行计算：

([A,0]+[C,0]+…)+([0,B]+[0,D]+…) [1]

然后再把高位字节产生的进位加到低字节，把低字节累加产生的进位加到高位字节，即可得到累加和。

b) 字节顺序的自主性：

若前面校验数据的次序反转为：

[B,A]+[D,C]+…+[Z,Y]

=([B,0]+[D,0]+…+[Z,0])+([0,A]+[0,C]+…+[0,Y])

其计算结果刚好就是[1]式计算结果的反转，原因在于两种排序产生的进位是相同的，因此，改变字节顺序，并不会改变它们的内在联系。

c) 并行计算

若 CPU 的字长是 16 的整数倍，那么，可以对数据进行并行计算，以加快计算的速度，而没有必要按照顺序逐字节或逐 16 位相加。

比如 CPU 字长为 32 位，那么在累加时可以这样并行计算：

[A,B,C,D]+[E,F,G,H]+…+[W,X,Y,Z]

计算结束后，再 32 位的计算结果进行折叠即可得到 16 位的运算结果。

也可以按照：

[D,C,B,A]+… OR [B,A,D,C]+…

然后再交换累加和字节序来得到正确结果。

（3）解析 ICMP 包

所谓解析 ICMP 包就是将收到的 ICMP 回送应答包的带来的各种信息在屏幕上输出显示，我们关注的信息主要包括：ICMP 报长度（是否与发送的回送请求包长度一致）；从发送请求到收到应答经历的时间（RTT），可以用收到应答包时的时间减去包中的时间戳值得到。

在解析以上信息前必须先判断所收到的应答包是否与所发送的请求包相对应，这可以通过验证收到标识符和序列号是否与请求包的一致来实现。

在读取 ICMP 包头时应注意，接收到的数据中包含 IP 头，要根据 IP 头中的头长字段来计算 IP 头的长度，接收缓冲区首地址加 IP 头的长度才是 ICMP 头的起始地址。

四、实验指导

本程序采用定时信号来周期性地启动发送线程来进行 ICMP 报文的发送；在主线程中则通过一个循环程序接收 ICMP 报文，并分析接收到的报文是否是本程序期待接收的 ICMP 响应报文，若是，则计算出 RTT。参考代码如下：

1、头文件中主要包含一些全局变量的声明、数据结构的声明、函数原型声明等。其内容列举如下：

```
//文件名：ping.h
#define ICMP_ECHOREPLY 0          /*Echo 应答*/
#define ICMP_ECHO      8          /*Echo 请求*/
#define BUFSIZE        1500       /*发送缓存最大值*/
#define DEFAULT_LEN 56            /*ping消息数据默认大小*/
typedef unsigned char u8;
typedef unsigned short u16;
typedef unsigned int u32;

/* ICMP消息头部*/
struct icmphdr {
```

```
    u8    type;
    u8    code;
    u16   checksum;
    union {
        struct {
            u16   id;
            u16   sequence;
        } echo;
        u32   gateway;
        struct {
            u16   unused;
            u16   mtu;
        } frag;                          /*pmtu发现*/
    }un;
    /*ICMP数据占位符*/
    u8    data[0];
#define icmp_id     un.echo.id
#define icmp_seq    un.echo.sequence
};
#define ICMP_HSIZE     sizeof(struct icmphdr)

char     *hostname;                      /*被ping的主机名*/
intdatalen = DEFAULT_LEN;                /*ICMP消息携带的数据长度*/
char     sendbuf [BUFSIZE];
char     recvbuf [BUFSIZE];

intnsent;                                /*发送的ICMP消息序号*/
intnrecv;
pid_t    pid;                            /*ping程序的进程PID*/
struct timeval recvtime;                 /*收到ICMP应答的时间戳*/
intsockfd;
struct sockaddr_in   dest;               /*目标主机的IP地址*/
struct sockaddr_in from;                 /* 发送ping应答消息的主机IP地址*/
struct sigaction       act_alarm;
struct sigaction       act_int;

/*函数原型*/
void alarm_handler(int);                 /*SIGALRM处理程序*/
void int_handler(int);                   /*SIGINT处理程序*/
void send_ping();                        /*发送ping消息程序*/
void recv_reply();                       /*接收ping应答程序*/
u16 checksum(u8 *buf, int len);          /*计算校验和程序*/
int handle_pkt();                        /*处理ping应答消息程序*/
void get_statistics (int, int);          /*统计ping命令的检测结果*/
```

```
void bail(const char *);                    /*错误报告*/
```

2、实现文件内容是各函数的具体实现。其内容如下：

```
//文件名：ping.c
#include<stdio.h>
#include<stdlib.h>
#include<sys/time.h>
#include<string.h>
#include<unistd.h>
#include<sys/socket.h>
#include<sys/types.h>
#include<netdb.h>
#include<errno.h>
#include<arpa/inet.h>
#include<signal.h>
#include<netinet/in.h>

#include"ping.h"

struct itimerval val_alarm = {
        .it_interval.tv_sec = 1,
        .it_interval.tv_usec = 0,
        .it_value.tv_sec = 0,
        .it_value.tv_usec = 1
        };

int main(int argc,char **argv)
{
   struct hostent *host;
   if (argc < 2) {
        printf("Usuage: %s hostname\n", argv[0]);
        exit(1);
   }
   if ((host = gethostbyname (argv[1])) == NULL) {
        perror ("can not understand the host name!\n");
        exit(1);
   }
   hostname = argv[1];
   memset(&dest, 0, sizeof(dest));
   dest.sin_family = PF_INET;
   dest.sin_port = ntohs(0);
   dest.sin_addr = *(struct in_addr *) host->h_addr_list[0];
   if((sockfd = socket (PF_INET, SOCK_RAW, IPPROTO_ICMP)) < 0) {
        perror ("raw socket created error!\n");
```

```
        exit(1);
    }
    pid = getpid();
    set_sighandler();
    printf("Ping %s(%s): %d bytes data in ICMP packets.\n\n",argv[1], inet_ntoa(dest. sin_addr), datalen);
    if((setitimer(ITIMER_REAL,&val_alarm,NULL)) == -1)
        bail("setitimer fails.");
 /*接收程序发出的ping命令的应答*/
    recv_reply();
    return 0;
}

void send_ping(void)
{
    struct icmphdr    *icmp_hdr;
    int    len;
    icmp_hdr = (struct icmphdr *) sendbuf;
    icmp_hdr->type = ICMP_ECHO;
    icmp_hdr->code = 0;
    icmp_hdr->icmp_id = pid;
    icmp_hdr->icmp_seq = nsent++;
    memset(icmp_hdr->data, 0xff, datalen);        /*ICMP数据部分填充模式为0xff*/
    gettimeofday((struct timeval*)icmp_hdr->data, NULL);
    len = ICMP_HSIZE + datalen;                   /*ICMP报文总长度*/
    icmp_hdr->checksum =0;                        /*清空校验和字段*/
    icmp_hdr->checksum = checksum((u8 *)icmp_hdr, len);
    sendto (sockfd, sendbuf, len, 0, (struct sockaddr *) &dest, sizeof(dest));
}

void recv_reply()
{
    int n, len;
    n = nrecv = 0;
    len = sizeof(from);
    while(nrecv < 3) {
        if((n = recvfrom(sockfd,  recvbuf,  sizeof(recvbuf), 0, (struct sockaddr*) &from, &len)) < 0) {
            if (errno= EINTR)
                continue;
            bail("recvfrom error");
        }
        gettimeofday(&recvtime, NULL);        /*记录收到应答的时间*/
        if (handle_pkt())                     /*接收到错误的ICMP应答消息*/
            continue;
        nrecv++;
```

```
    }
    get_statistics (nsent, nrecv);
}

u16 checksum(u8 *buf, int len)
{
    int sum = 0;
    u16 *cbuf;
    cbuf = (u16*) buf;
    while (len > 1) {
        sum += *cbuf++;
        len -= 2;
    }
    if(len)
        sum += *(u8 *)cbuf;
    sum = (sum >>16) + (sum & 0xffff);
    sum += (sum >> 16);
    return ~sum;
}

int handle_pkt()
{
    struct iphdr*ip;
    struct icmphdr    *icmp;
    int    ip_hlen;
    u16   ip_datalen;
    double      rtt;
    struct timeval     *sendtime;
    ip = (struct iphdr *)recvbuf;
    ip_hlen = ip->hlen << 2;                    /* 获得IP报文头部长度*/
    ip_datalen = ntohs(ip->tot_len) - ip_hlen;
    /*获得ICMP消息头部指针*/
    icmp = (struct icmphdr *) (recvbuf + ip_hlen);
    if(checksum(( u8*) icmp, ip_datalen))
        return -1;
    /*检查该ICMP报文是否是期望接收的*/
    if ( icmp->icmp_id != pid)
        return -1;
    sendtime = (struct timeval *)icmp->data;
    rtt = ((&recvtime)->tv_sec - sendtime->tv_sec)*1000 + ((&recvtime) ->tv_usec - sendtime-
>tv_usec)/1000.0;
    printf ("%d bytes from %s: icmp_seq = %u ttl=%d rtt=%.6f ms\n", ip_datalen, inet_ntoa(from. sin_addr),
icmp->icmp_seq, ip->ttl);
    return 0;
```

```
}

void  set_sighandler( )
{
    act_alarm.sa_handler = alarm_handler;
    if (sigaction(SIGALRM, &act_alarm, NULL) == -1)
            bail("SIGALRM handler setting fails.");
    act_int.sa_handler = int_handler;
    if(sigaction(SIGINT,&act_int, NULL) == -1)
            bail("SIGINT handler setting fails.");
}

void  get_statistics (int  nsent, int  nrecv)
{
    printf (" ---%s ping  statistics  ---\n", inet_ntoa(dest.sin_addr));
    printf (" %d  packets  transmitted, %d received, %0.0f packet  loss\n", nsent, nrecv, 1.0*( nsent  -  nrecv)
/nsent * 100);
}

void bail(const char *on_what)
{
    fputs (strerror ( errno), stderr);
    fputs ( ":", stderr);
    fputs (on_what, stderr);
    fputc ('\n', stderr);
    exit (1);
}

void int_handler(int sig)
{
    get_statistics (nsent, nrecv);
    close (sockfd);
    exit (1);
}

void alarm_handler(int sig)
{
    send_ping();
}
```